

(1) Working with files and directories

Inside the lab2 directory, creating files, data , output directories

```
ibab@IBAB-Workshop-Comp15:~/lab2$ ls -l
total 4
drwxrwxr-x 3 ibab ibab 4096 Sep 21 14:20 lab2
ibab@IBAB-Workshop-Comp15:~/lab2$ mkdir files
ibab@IBAB-Workshop-Comp15:~/lab2$ mkdir data
ibab@IBAB-Workshop-Comp15:~/lab2$ mkdir output
ibab@IBAB-Workshop-Comp15:~/lab2$ ls
data files lab2 output
```

Using touch command creating different file texts and changing file name using mv command

```
ibab@IBAB-Workshop-Comp15:~/lab2$ cd files
ibab@IBAB-Workshop-Comp15:~/lab2/files$ touch file1.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ touch file2.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ touch file3.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ mv file1.txt file.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls
file.txt file2.txt file3.txt
```

Contents of lab2 directory using ls -l command

d rwx rwx r-x is the permissions of file

d= directory

r= read

w= write

x= execute

First rwx = The owner has the read,write and execute permission

Second rwx = Group have the read,write and execute permission

Third r-x = Other have the read and execute permission only

```
ibab@IBAB-Workshop-Comp15:~/lab2$ ls -l
total 16
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:28 data
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:32 files
drwxrwxr-x 3 ibab ibab 4096 Sep 21 14:20 lab2
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:28 output
ibab@IBAB-Workshop-Comp15:~/lab2$
```

2.Viewing files

cat command

```

The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.
Prokaryotes (Bacteria)

In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.

    The Shine-Dalgarno (SD) Sequence: The mRNA transcript contains a specific purine-rich consensus sequence (typically AGGAGG) located about 8 to 10 nucleotides upstream (5' side) of the start codon.

    16S rRNA Pairing: The small ribosomal subunit (30S) contains 16S ribosomal RNA, which possesses a complementary sequence at its 3' end (the anti-Shine-Dalgarno sequence).

    Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downstream AUG codon drops perfectly into the ribosome's P-site, ready for the initiator tRNA (fMet-tRNA) to bind.
    Eukaryotes (Plants and Animals)

Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, they locate the start codon through a physical "scanning" mechanism originating from the physical end of the mRNA transcript.

    5' Cap Recognition: The small ribosomal subunit (40S), already carrying the initiator tRNA (Met-tRNAi) and various initiation factors, binds to the specialized 5' cap structure (m7G) at the extreme 5' end of the mRNA.

    Scanning: Powered by ATP, the 40S complex slides down the mRNA in the 5' to 3' direction, reading the sequence as it goes.

    The Kozak Sequence: The ribosome is generally looking for the first AUG it encounters, but its efficiency in locking on depends on the surrounding nucleotides, known as the Kozak consensus sequence (typically GCCRCCAUUG, where R is a purine like A or G).

    Initiation: When the anticodon of the initiator tRNA perfectly matches an AUG embedded in a strong Kozak context, the scanning mechanism halts. The large ribosomal subunit (60S) then joins to form the complete 80S ribosome, and translation begins. (If the first AUG is in a very weak context, the ribosome may occasionally skip it and initiate at the next AUG, a phenomenon known as "leaky scanning").

    Ribosome skipping sequences, most commonly known as 2A peptides (or CHYSELS - cis-acting hydrolase elements), are short amino acid sequences that force the ribosome to produce two distinct, disconnected proteins from a single continuous mRNA transcript.

Originally discovered in the Foot-and-mouth disease virus, these sequences are now heavily utilized in biotechnology and gene therapy to co-express multiple genes simultaneously without needing multiple promoters.
The Mechanism: A Misnomer

The term "skipping" is actually a misnomer. The ribosome does not skip over any mRNA codons, nor does an enzyme come along later to cleave a large protein into two smaller ones. Instead, 2A peptides cause a highly specific translational recoding event—a failure of peptide bond formation.

All functional 2A peptides share a conserved structural motif at their C-terminus: DXENPQP (Asp-X-Glu-X-Asn-Pro-Gly-Pro).

    Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing the peptide normally.

    The Jam: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).

    The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.

    Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).
Sample.txt (END)

```

cat command: It shows the whole file

```

lsab@IRAB-Workshop-Comp11:~/lab2/Files$ more Sample.txt
The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.
Prokaryotes (Bacteria)

In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.

    The Shine-Dalgarno (SD) Sequence: The mRNA transcript contains a specific purine-rich consensus sequence (typically AGGAGG) located about 8 to 10 nucleotides upstream (5' side) of the start codon.

    16S rRNA Pairing: The small ribosomal subunit (30S) contains 16S ribosomal RNA, which possesses a complementary sequence at its 3' end (the anti-Shine-Dalgarno sequence).

    Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downstream AUG codon drops perfectly into the ribosome's P-site, ready for the initiator tRNA (fMet-tRNA) to bind.
    Eukaryotes (Plants and Animals)

Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, they locate the start codon through a physical "scanning" mechanism originating from the physical end of the mRNA transcript.

    5' Cap Recognition: The small ribosomal subunit (40S), already carrying the initiator tRNA (Met-tRNAi) and various initiation factors, binds to the specialized 5' cap structure (m7G) at the extreme 5' end of the mRNA.

    Scanning: Powered by ATP, the 40S complex slides down the mRNA in the 5' to 3' direction, reading the sequence as it goes.

    The Kozak Sequence: The ribosome is generally looking for the first AUG it encounters, but its efficiency in locking on depends on the surrounding nucleotides, known as the Kozak consensus sequence (typically GCCRCCAUUG, where R is a purine like A or G).

    Initiation: When the anticodon of the initiator tRNA perfectly matches an AUG embedded in a strong Kozak context, the scanning mechanism halts. The large ribosomal subunit (60S) then joins to form the complete 80S ribosome, and translation begins. (If the first AUG is in a very weak context, the ribosome may occasionally skip it and initiate at the next AUG, a phenomenon known as "leaky scanning").

    Ribosome skipping sequences, most commonly known as 2A peptides (or CHYSELS - cis-acting hydrolase elements), are short amino acid sequences that force the ribosome to produce two distinct, disconnected proteins from a single continuous mRNA transcript.

Originally discovered in the Foot-and-mouth disease virus, these sequences are now heavily utilized in biotechnology and gene therapy to co-express multiple genes simultaneously without needing multiple promoters.
The Mechanism: A Misnomer

The term "skipping" is actually a misnomer. The ribosome does not skip over any mRNA codons, nor does an enzyme come along later to cleave a large protein into two smaller ones. Instead, 2A peptides cause a highly specific translational recoding event—a failure of peptide bond formation.

All functional 2A peptides share a conserved structural motif at their C-terminus: DXENPQP (Asp-X-Glu-X-Asn-Pro-Gly-Pro).

    Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing the peptide normally.

    The Jam: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).

    The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.

    Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```

more : command

```

lsab@IRAB-Workshop-Comp11:~/lab2/Files$ head -n 5 Sample.txt
The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.
Prokaryotes (Bacteria)

In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.

```

Head : It shows first few lines of the file, we can mention the number of lines

```

lsab@IRAB-Workshop-Comp11:~/lab2/Files$ tail -n 5 Sample.txt
    The Jam: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).

    The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.

    Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```

Tail: It shows last few lines of the file , we can mention the number of lines\

Less : Provides the file in compact view

The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.

Prokaryotes (Bacteria)

In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.

The Shine-Dalgarno (SD) Sequence: The mRNA transcript contains a specific purine-rich consensus sequence (typically AGGAGG) located about 8 to 10 nucleotides upstream (5' side) of the start codon.

16S rRNA Pairing: The small ribosomal subunit (30S) contains 16S ribosomal RNA, which possesses a complementary sequence at its 3' end (the anti-Shine-Dalgarno sequence).

Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downstream AUG codon drops perfectly into the ribosome's P-site, ready for the initiator tRNA (fMet-tRNA) to bind.

Eukaryotes (Plants and Animals)

Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, they locate the start codon through a physical "scanning" mechanism originating from the physical end of the mRNA transcript.

5' Cap Recognition: The small ribosomal subunit (40S), already carrying the initiator tRNA (Met-tRNAi) and various initiation factors, binds to the specialized 5' cap structure (m7G) at the extreme 5' end of the mRNA.

Scanning: Powered by ATP, the 40S complex slides down the mRNA in the 5' to 3' direction, reading the sequence as it goes.

The Kozak Sequence: The ribosome is generally looking for the first AUG it encounters, but its efficiency in locking on depends on the surrounding nucleotides, known as the Kozak consensus sequence (typically gccRccAUGG, where R is a purine like A or G).

Initiation: When the anticodon of the initiator tRNA perfectly matches an AUG embedded in a strong Kozak context, the scanning mechanism halts. The large ribosomal subunit (60S) then joins to form the complete 80S ribosome, and translation begins. (If the first AUG is in a very weak context, the ribosome may occasionally skip it and initiate at the next AUG, a phenomenon known as "leaky scanning").

Ribosome skipping sequences, most commonly known as 2A peptides (or CHYSELs - cis-acting hydrolyase elements), are short amino acid sequences that force the ribosome to produce two distinct, disconnected proteins from a single continuous mRNA transcript.

Originally discovered in the Foot-and-mouth disease virus, these sequences are now heavily utilized in biotechnology and gene therapy to co-express multiple genes simultaneously without needing multiple promoters.

The Mechanism: A Misnomer

The term "skipping" is actually a misnomer. The ribosome does not skip over any mRNA codons, nor does an enzyme come along later to cleave a large protein into two smaller ones. Instead, 2A peptides cause a highly specific translational recoding event—a failure of peptide bond formation.

All Functional 2A peptides share a conserved structural motif at their C-terminus: DxExMPGP (Asp-X-Glu-X-Asn-Pro-Gly-Pro).

Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing the peptide normally.

The Jan: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).

The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.

Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Proline-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

Sample.txt (END)

(3) Moving and copying files

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cp files.txt /home/ibab/lab2/output/
cp: cannot stat 'files.txt': No such file or directory
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cp file.txt /home/ibab/lab2/output/
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cd ..
ibab@IBAB-Workshop-Comp15:~/lab2$ ls
data  files  output
ibab@IBAB-Workshop-Comp15:~/lab2$ cd output
ibab@IBAB-Workshop-Comp15:~/lab2/output$ ls
file.txt
```

Copying file.txt from files to output

```
ibab@IBAB-Workshop-Comp15:~/lab2/output$ mv file.txt file_copy.txt
ibab@IBAB-Workshop-Comp15:~/lab2/output$ ls
file_copy.txt
```

Renaming the file name using mv command

```

ibab@IBAB-Workshop-Comp15:~/lab2/output$ mv file_copy.txt /home/ibab/lab2/files
ibab@IBAB-Workshop-Comp15:~/lab2/output$ cd..
cd..: command not found
ibab@IBAB-Workshop-Comp15:~/lab2/output$ cd ..
ibab@IBAB-Workshop-Comp15:~/lab2$ cd files
ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls
Sample.txt  file.txt  file2.txt  file3.txt  file_copy.txt  outout  output

```

Moving back the file from output to files using mv command

```

ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls
Sample.txt  file.txt  file2.txt  file3.txt  file_copy.txt  outout  output
ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls -l
total 8
-rw-rw-r-- 1 ibab ibab 4145 Sep 21 14:51 Sample.txt
-rw-rw-r-- 1 ibab ibab   0 Sep 21 14:31 file.txt
-rw-rw-r-- 1 ibab ibab   0 Sep 21 14:31 file2.txt
-rw-rw-r-- 1 ibab ibab   0 Sep 21 14:31 file3.txt
-rw-rw-r-- 1 ibab ibab   0 Sep 21 17:27 file_copy.txt
-rw-rw-r-- 1 ibab ibab   0 Sep 21 17:24 outout
-rw-rw-r-- 1 ibab ibab   0 Sep 21 17:25 output

```

Verified location of files and details

cp is used for copying the files within a folder or outside , where two or more files are created and original file is retained

```

ibab@IBAB-Workshop-Comp15:~/lab2/files$ cp files.txt /home/ibab/lab2/output/
cp: cannot stat 'files.txt': No such file or directory
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cp file.txt /home/ibab/lab2/output/
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cd ..
ibab@IBAB-Workshop-Comp15:~/lab2$ ls
data  files  output
ibab@IBAB-Workshop-Comp15:~/lab2$ cd output
ibab@IBAB-Workshop-Comp15:~/lab2/output$ ls
file.txt

```

mv is move command it is used for moving the file from one folder to other, no multiple same files are created original file is moved. mv command is also used for renaming the files in the linux.

```

ibab@IBAB-Workshop-Comp15:~/lab2/output$ mv file_copy.txt /home/ibab/lab2/files
ibab@IBAB-Workshop-Comp15:~/lab2/output$ cd..
cd..: command not found
ibab@IBAB-Workshop-Comp15:~/lab2/output$ cd ..
ibab@IBAB-Workshop-Comp15:~/lab2$ cd files
ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls
Sample.txt  file.txt  file2.txt  file3.txt  file_copy.txt  outout  output

```

```

ibab@IBAB-Workshop-Comp15:~/lab2/output$ mv file.txt file_copy.txt
ibab@IBAB-Workshop-Comp15:~/lab2/output$ ls
file_copy.txt

```

(4) Finding information using grep

```

ibab@IBAB-Workshop-Comp15:~/lab2/files$ grep "Glycine" Sample.txt
The Break: When the ribosome attempts to form a peptide bond between the final glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.
Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin
e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```

Multiple times occurring word

```

ibab@IBAB-Workshop-Comp15:~/lab2/files$ grep "ribosome" Sample.txt
The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.
In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.
Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downstream AUG codon drops perfectly into the ribosome's P-site, ready for the initiator tRNA (fMet-tRNA) to bind.
Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, they locate the start codon through a physical "scanning" mechanism originating from the physical end of the mRNA transcript.
The Kozak Sequence: The ribosome is generally looking for the first AUG it encounters, but its efficiency in locking on depends on the surrounding nucleotides, known as the Kozak consensus sequence (typically gccRccAUGG, where R is a purine like A or G).
Initiation: When the anticodon of the initiator tRNA perfectly matches an AUG embedded in a strong Kozak context, the scanning mechanism halts. The large ribosomal subunit (60S) then joins to form the complete 80S ribosome, and translation begins. (If the first AUG is in a very weak context, the ribosome may occasionally skip it and initiate at the next AUG, a phenomenon known as "leaky scanning").
Ribosome skipping sequences, most commonly known as 2A peptides (or CHYSELS - cis-acting hydrolase elements), are short amino acid sequences that force the ribosome to produce two distinct, disconnected proteins from a single continuous mRNA transcript.
The term "skipping" is actually a misnomer. The ribosome does not skip over any mRNA codons, nor does an enzyme come along later to cleave a large protein into two smaller ones. Instead, 2A peptides cause a highly specific translational recoding event—a failure of peptide bond formation.
Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing the peptide normally.
The Jan: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).
The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.
Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin
e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```

grep -i command

```

ibab@IBAB-Workshop-Comp15:~/lab2/files$ grep -i "Ribosome" Sample.txt
The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.
In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.
Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downstream AUG codon drops perfectly into the ribosome's P-site, ready for the initiator tRNA (fMet-tRNA) to bind.
Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, they locate the start codon through a physical "scanning" mechanism originating from the physical end of the mRNA transcript.
The Kozak Sequence: The ribosome is generally looking for the first AUG it encounters, but its efficiency in locking on depends on the surrounding nucleotides, known as the Kozak consensus sequence (typically gccRccAUGG, where R is a purine like A or G).
Initiation: When the anticodon of the initiator tRNA perfectly matches an AUG embedded in a strong Kozak context, the scanning mechanism halts. The large ribosomal subunit (60S) then joins to form the complete 80S ribosome, and translation begins. (If the first AUG is in a very weak context, the ribosome may occasionally skip it and initiate at the next AUG, a phenomenon known as "leaky scanning").
Ribosome skipping sequences, most commonly known as 2A peptides (or CHYSELS - cis-acting hydrolase elements), are short amino acid sequences that force the ribosome to produce two distinct, disconnected proteins from a single continuous mRNA transcript.
The term "skipping" is actually a misnomer. The ribosome does not skip over any mRNA codons, nor does an enzyme come along later to cleave a large protein into two smaller ones. Instead, 2A peptides cause a highly specific translational recoding event—a failure of peptide bond formation.
Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing the peptide normally.
The Jan: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).
The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.
Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin
e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```

Use of grep for Word does not occur in the file

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ grep "replication" Sample.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$
```

grep command is used to search particular word in the text files, we can also look for a word occurring multiple times in the text files

(5) Finding files using find

Finding all files with the .txt extension.

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ find -name "*.txt"
./file3.txt
./file2.txt
./Sample.txt
./file.txt
./file_copy.txt
```

Find the file named sample.txt.

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ find . -name "Sample.txt"
./Sample.txt
```

Find all the directories in lab 2

```
ibab@IBAB-Workshop-Comp15:~/lab2$ find . -type d
.
./data
./files
./output
```

Grep searches for patterns in a file where as find command helps to search for files and directories based on type ,name , property and size.

(6) Basic text processing

```
ibab@IBAB-Workshop-Comp15:~/lab2$ cd files
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -l Sample.txt
37 Sample.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -w Sample.txt
618 Sample.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -c Sample.txt
4145 Sample.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc Sample.txt
 37  618 4145 Sample.txt
```

Text processing using wc -l, wc -w, wc -c, wc


```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ sort Sample.txt
```

16S rRNA Pairing: The small ribosomal subunit (30S) contains 16S ribosomal RNA, which possesses a complementary sequence at its 3' end (the anti-Shine-Dalgarno sequence).

5' Cap Recognition: The small ribosomal subunit (40S), already carrying the Initiator tRNA (Met-tRNA_i) and various initiation factors, binds to the specialized 5' cap structure (m⁷G) at the extreme 5' end of the mRNA.

All Functional 2A peptides share a conserved structural motif at their C-terminus: DxEXNPGP (Asp-X-Glu-X-Asn-Pro-Gly-Pro).

Eukaryotes (Plants and Animals)

Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, they locate the start codon through a physical "scanning" mechanism originating from the physical end of the mRNA transcript.

In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.

Initiation: When the anticodon of the Initiator tRNA perfectly matches an AUG embedded in a strong Kozak context, the scanning mechanism halts. The large ribosomal subunit (60S) then joins to form the complete 80S ribosome, and translation begins. (If the first AUG is in a very weak context, the ribosome may occasionally skip it and initiate at the next AUG, a phenomenon known as "leaky scanning").

Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downstream AUG codon drops perfectly into the ribosome's P-site, ready for the Initiator tRNA (Met-tRNA_i) to bind.

Originally discovered in the Foot-and-mouth disease virus, these sequences are now heavily utilized in biotechnology and gene therapy to co-express multiple genes simultaneously without needing multiple promoters.

Prokaryotes (Bacteria)

Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in Glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Prolin e-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

Ribosome skipping sequences, most commonly known as 2A peptides (or CHYSELs - cis-acting hydrolase elements), are short amino acid sequences that force the ribosome to produce two distinct, disconnected proteins from a single continuous mRNA transcript.

Scanning: Powered by ATP, the 40S complex slides down the mRNA in the 5' to 3' direction, reading the sequence as it goes.

The Break: When the ribosome attempts to form a peptide bond between the final Glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.

The Jam: As the 2A peptide is synthesized, its specific shape inside the ribosome's exit tunnel interacts with the ribosome core. This alters the geometry of the peptidyl-transferase center (the part of the ribosome that links amino acids together).

The Kozak Sequence: The ribosome is generally looking for the first AUG it encounters, but its efficiency in locking on depends on the surrounding nucleotides, known as the Kozak consensus sequence (typically gccRccAUG, where R is a purine like A or G).

The Mechanism: A Misnomer

The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs fundamentally between prokaryotes and eukaryotes.

The Shine-Dalgarno (SD) Sequence: The mRNA transcript contains a specific purine-rich consensus sequence (typically AGGAGG) located about 8 to 10 nucleotides upstream (5' side) of the start codon.

The term "skipping" is actually a misnomer. The ribosome does not skip over any mRNA codons, nor does an enzyme come along later to cleave a large protein into two smaller ones. Instead, 2A peptides cause a highly specific translational recoding event—a failure of peptide bond formation.

Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing the peptide normally.

Command sort for the file

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ nano second.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat second.txt
rohit
rohit
rohit
jivitesh
jivitesh
yugashree
vaishnavi
vaishnavi
pal
debjaani
debjaani
yugashree
ibab@IBAB-Workshop-Comp15:~/lab2/files$ uniq second.txt
rohit
jivitesh
yugashree
vaishnavi
pal
debjaani
yugashree
ibab@IBAB-Workshop-Comp15:~/lab2/files$
```

uniq command for removing consecutive repeated lines

wc command used for word count

sort command Reorders the lines of a text file in a specified sequence

uniq command Filters out or reports repeated, adjacent lines in a text file.

(7) Output redirection using >

```
ibab@IBAB-Workshop-Comp15:~$ pwd > location.txt
ibab@IBAB-Workshop-Comp15:~$ ls -l >listing.txt
ibab@IBAB-Workshop-Comp15:~$ cat listing.txt
total 96
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 2
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Desktop
drwxr-xr-x 5 ibab ibab 4096 Sep 18 12:30 Docs
drwxr-xr-x 5 ibab ibab 4096 Sep 18 10:43 Documents
drwxr-xr-x 2 ibab ibab 4096 Sep 21 11:42 Downloads
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Music
drwxr-xr-x 3 ibab ibab 4096 Sep 17 14:38 Pictures
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Public
drwxrwxr-x 3 ibab ibab 4096 Sep 17 15:25 R
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Templates
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Videos
-rw-rw-r-- 1 ibab ibab 279 Sep 17 14:25 apt.conf
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 data
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:38 directory.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 18 13:01 file1_hardlink
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:25 files
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 lab
drwxrwxr-x 5 ibab ibab 4096 Sep 21 14:51 lab2
-rw-rw-r-- 1 ibab ibab 0 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 output
drwxrwxr-x 3 ibab ibab 4096 Sep 21 15:34 pushtestdir
drwxrwxr-x 4 ibab ibab 4096 Sep 21 16:23 sails_python_labs
drwx----- 7 ibab ibab 4096 Sep 21 11:39 snap
-rw-rw-r-- 3 ibab ibab 771 Sep 18 12:17 testdir-hardlink
lrwxrwxrwx 1 ibab ibab 5 Sep 18 12:46 testdir-symlink -> file1
ibab@IBAB-Workshop-Comp15:~$
```

```
ibab@IBAB-Workshop-Comp15:~$ ls > directory.txt
ibab@IBAB-Workshop-Comp15:~$ cat directory.txt
2
Desktop
Docs
Documents
Downloads
Music
Pictures
Public
R
Templates
Videos
apt.conf
data
directory.txt
file1_hardlink
files
lab
lab2
output
pushtestdir
sails_python_labs
snap
testdir-hardlink
testdir-symlink
```


Purpose of > in command line is to redirect the output of a command , write in a text file.
When > is used to output for a file , it over writes the contents of the file

Example: I created a file named overwrite.txt and added few my information ,when I used it for > symbol it overwrites on existing information in file.

```
ibab@IBAB-Workshop-Comp15:~$ touch overwrite.txt
ibab@IBAB-Workshop-Comp15:~$ cat overwrite.txt
I am Rohit
Enrolled in SAILS batch 2
Having technical classes at IBAB
ibab@IBAB-Workshop-Comp15:~$ ls -l > overwrite.txt
ibab@IBAB-Workshop-Comp15:~$ cat overwrite.txt
total 100
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 2
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Desktop
drwxr-xr-x 5 ibab ibab 4096 Sep 18 12:30 Docs
drwxr-xr-x 5 ibab ibab 4096 Sep 18 10:43 Documents
drwxr-xr-x 2 ibab ibab 4096 Sep 21 11:42 Downloads
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Music
drwxr-xr-x 3 ibab ibab 4096 Sep 17 14:38 Pictures
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Public
drwxrwxr-x 3 ibab ibab 4096 Sep 17 15:25 R
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Templates
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Videos
-rw-rw-r-- 1 ibab ibab 279 Sep 17 14:25 apt.conf
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 data
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:38 directory.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 18 13:01 file1_hardlink
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:25 files
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 lab
drwxrwxr-x 5 ibab ibab 4096 Sep 21 14:51 lab2
-rw-rw-r-- 1 ibab ibab 1318 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 output
-rw-rw-r-- 1 ibab ibab 0 Sep 22 11:50 overwrite.txt
drwxrwxr-x 3 ibab ibab 4096 Sep 21 15:34 pushtestdir
drwxrwxr-x 4 ibab ibab 4096 Sep 21 16:23 sails_python_labs
drwx----- 7 ibab ibab 4096 Sep 21 11:39 snap
-rw-rw-r-- 3 ibab ibab 771 Sep 18 12:17 testdir-hardlink
lrwxrwxrwx 1 ibab ibab 5 Sep 18 12:46 testdir-symlink -> file1
ibab@IBAB-Workshop-Comp15:~$
```

(8) Appending output using >>

```
ibab@IBAB-Workshop-Comp15:~$ pwd > information.txt
ibab@IBAB-Workshop-Comp15:~$ cat information.txt
/home/ibab
ibab@IBAB-Workshop-Comp15:~$ whoami >> information.txt
ibab@IBAB-Workshop-Comp15:~$ cat information.txt
/home/ibab
ibab
ibab@IBAB-Workshop-Comp15:~$ hostname >> information.txt
ibab@IBAB-Workshop-Comp15:~$ cat information.txt
/home/ibab
ibab
IBAB-Workshop-Comp15.ibab.ac.in
ibab@IBAB-Workshop-Comp15:~$
```

The difference between > and >> is , > it overwrites the information in the file which leads to erase/delete of previous contents of the file. Whereas >> , it appends the new output with existing information at the end.

```
ibab@IBAB-Workshop-Comp15:~$ touch overwrite.txt
ibab@IBAB-Workshop-Comp15:~$ cat overwrite.txt
I am Rohit
Enrolled in SAILS batch 2
Having technical classes at IBAB
ibab@IBAB-Workshop-Comp15:~$ ls -l > overwrite.txt
ibab@IBAB-Workshop-Comp15:~$ cat overwrite.txt
total 100
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 2
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Desktop
drwxr-xr-x 5 ibab ibab 4096 Sep 18 12:30 Docs
drwxr-xr-x 5 ibab ibab 4096 Sep 18 10:43 Documents
drwxr-xr-x 2 ibab ibab 4096 Sep 21 11:42 Downloads
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Music
drwxr-xr-x 3 ibab ibab 4096 Sep 17 14:30 Pictures
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Public
drwxrwxr-x 3 ibab ibab 4096 Sep 17 15:25 R
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Templates
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Videos
-rw-rw-r-- 1 ibab ibab 279 Sep 17 14:25 apt.conf
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 data
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:30 directory.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 18 13:01 file1_hardlink
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:25 files
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 lab
drwxrwxr-x 5 ibab ibab 4096 Sep 21 14:51 lab2
-rw-rw-r-- 1 ibab ibab 1318 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 output
-rw-rw-r-- 1 ibab ibab 0 Sep 22 11:50 overwrite.txt
drwxrwxr-x 3 ibab ibab 4096 Sep 21 15:34 pushtestdir
drwxrwxr-x 4 ibab ibab 4096 Sep 21 16:23 sails_python_labs
drwx----- 7 ibab ibab 4096 Sep 21 11:39 snap
-rw-rw-r-- 3 ibab ibab 771 Sep 18 12:17 testdir-hardlink
lrwxrwxrwx 1 ibab ibab 5 Sep 18 12:46 testdir-symlink -> file1
ibab@IBAB-Workshop-Comp15:~$
```

```
ibab@IBAB-Workshop-Comp15:~$ pwd > information.txt
ibab@IBAB-Workshop-Comp15:~$ cat information.txt
/home/ibab
ibab@IBAB-Workshop-Comp15:~$ whoami >> information.txt
ibab@IBAB-Workshop-Comp15:~$ cat information.txt
/home/ibab
ibab
ibab@IBAB-Workshop-Comp15:~$ hostname >> information.txt
ibab@IBAB-Workshop-Comp15:~$ cat information.txt
/home/ibab
ibab
IBAB-Workshop-Comp15.ibab.ac.in
ibab@IBAB-Workshop-Comp15:~$
```

(9) Input redirection using <

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -l < Sample.txt
37
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -l Sample.txt
37 Sample.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$
```

< command counts the number of lines in the file `Sample.txt` without printing the filename in the output.

(10) Using the pipe |

```
ibab@IBAB-Workshop-Comp15:~$ ls -l | more
total 108
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 2
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Desktop
drwxr-xr-x 5 ibab ibab 4096 Sep 18 12:30 Docs
drwxr-xr-x 5 ibab ibab 4096 Sep 18 10:43 Documents
drwxr-xr-x 2 ibab ibab 4096 Sep 21 11:42 Downloads
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Music
drwxr-xr-x 3 ibab ibab 4096 Sep 17 14:38 Pictures
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Public
drwxrwxr-x 3 ibab ibab 4096 Sep 17 15:25 R
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Templates
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Videos
-rw-rw-r-- 1 ibab ibab 279 Sep 17 14:25 apt.conf
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 data
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:38 directory.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 18 13:01 file1_hardlink
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:25 files
-rw-rw-r-- 1 ibab ibab 48 Sep 22 11:56 information.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 lab
drwxrwxr-x 5 ibab ibab 4096 Sep 21 14:51 lab2
-rw-rw-r-- 1 ibab ibab 1318 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 output
-rw-rw-r-- 1 ibab ibab 1374 Sep 22 11:50 overwrite.txt
drwxrwxr-x 3 ibab ibab 4096 Sep 21 15:34 pushtestdir
drwxrwxr-x 4 ibab ibab 4096 Sep 21 16:23 sails_python_labs
drwx----- 7 ibab ibab 4096 Sep 21 11:39 snap
-rw-rw-r-- 3 ibab ibab 771 Sep 18 12:17 testdir-hardlink
lrwxrwxrwx 1 ibab ibab 5 Sep 18 12:46 testdir-symlink -> file1
ibab@IBAB-Workshop-Comp15:~$ ls -l | grep ".txt"
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:38 directory.txt
-rw-rw-r-- 1 ibab ibab 48 Sep 22 11:56 information.txt
-rw-rw-r-- 1 ibab ibab 1318 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
-rw-rw-r-- 1 ibab ibab 1374 Sep 22 11:50 overwrite.txt
```

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat Sample.txt | grep "Glycine"
```

The Break: When the ribosome attempts to form a peptide bond between the final **Glycine** (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry. Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in **Glycine**, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Proline-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat Sample.txt | grep "word"
```

Use of grep command for “word” in a file using | in the command line

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat Sample.txt | wc -l
37
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -l Sample.txt
37 Sample.txt
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cd
ibab@IBAB-Workshop-Comp15:~$ ls | grep ".txt" | wc -l
5
```

- “|” operator is used to connect two or more commands, where it takes outputs of each command and provides input to next (from left side to right side).

(11) Combining multiple operators

- ❖ “>” is used for to redirect the output of a command, write in a text file. When > is used to output for a file, it overwrites the contents of the file
- ❖ “>>” used for to redirect the output of a command, write in a text file, it appends the new output with existing information at the end.
- ❖ “|” is used to connect two or more commands consecutively.

```
ibab@IBAB-Workshop-Comp15:~$ ls | grep ".txt" | wc -l
5
ibab@IBAB-Workshop-Comp15:~$ ls -l | grep ".txt" > textfiles.txt
ibab@IBAB-Workshop-Comp15:~$ cat textfiles.txt
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:38 directory.txt
-rw-rw-r-- 1 ibab ibab 48 Sep 22 11:56 information.txt
-rw-rw-r-- 1 ibab ibab 1318 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
-rw-rw-r-- 1 ibab ibab 1374 Sep 22 11:50 overwrite.txt
-rw-rw-r-- 1 ibab ibab 0 Sep 22 12:28 textfiles.txt
```

```

ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat Sample.txt | grep "Glycine" | wc -l
2
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat Sample.txt | grep "ribosome" | wc -l
12
ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls commands_output.txt
ls: cannot access 'commands_output.txt': No such file or directory
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cd
ibab@IBAB-Workshop-Comp15:~$ ls > commands_output.txt
ibab@IBAB-Workshop-Comp15:~$ pwd >> commands_output.txt
ibab@IBAB-Workshop-Comp15:~$ whoami >> commands_output.txt
ibab@IBAB-Workshop-Comp15:~$ cat commands_output.txt
2
Desktop
Docs
Documents
Downloads
Music
Pictures
Public
R
Templates
Videos
apt.conf
commands_output.txt
data
directory.txt
file1_hardlink
files
information.txt
lab
lab2
listing.txt
location.txt
output
overwrite.txt
pushtestdir
sails_python_labs
snap
testdir-hardlink
testdir-symlink
textfiles.txt
/home/ibab
ibab

```

(12) Exploring Linux as "everything is a file"

The “less” command helps for long terminal outputs where it prevents text scrolling off screen lets us to view page by page

```
processor      : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 191
model name    : Intel(R) Core(TM) i5-14500
stepping      : 2
microcode     : 0x3e
cpu MHz       : 800.242
cache size    : 24576 KB
physical id   : 0
siblings      : 20
core id       : 0
cpu cores     : 14
apicid        : 0
initial apicid : 0
fpu           : yes
fpu_exception : yes
cpuid level   : 32
wp            : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
nonstop_tsc   : yes
cpuid         : 0
aperfmpperf   : 1000000000
tsc_known_freq : 1000
pni           : yes
pclmulqdq     : 0x30
dtes64        : yes
monitor       : yes
ds_cpl        : yes
vmx           : yes
cstpr         : yes
cqm           : no
cqm_llh       : no
cqm_occup_llh : no
cqm_rdt       : no
rdt_occupancy : no
```

```
MemTotal: 15523492 kB
MemFree: 7972784 kB
MemAvailable: 11413012 kB
Buffers: 142032 kB
Cached: 3595388 kB
SwapCached: 0 kB
Active: 3513768 kB
Inactive: 2428340 kB
Active(anon): 2318496 kB
Inactive(anon): 0 kB
Active(file): 1195272 kB
Inactive(file): 2428340 kB
Unevictable: 843920 kB
Mlocked: 16 kB
SwapTotal: 8000508 kB
SwapFree: 8000508 kB
Zswap: 0 kB
Zswapped: 0 kB
Dirty: 1088 kB
Writeback: 0 kB
AnonPages: 3049164 kB
```



```

ibab@IBAB-Workshop-Comp15:~$ cat /proc/cpuinfo | less
ibab@IBAB-Workshop-Comp15:~$ cat /proc/cpuinfo | grep "model name"
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
model name      : Intel(R) Core(TM) i5-14500
ibab@IBAB-Workshop-Comp15:~$ cat /proc/meminfo | grep "KernelStack"
ibab@IBAB-Workshop-Comp15:~$ cat /proc/meminfo | grep "KernelStack"
KernelStack:    19984 kB
ibab@IBAB-Workshop-Comp15:~$

```

(13) Understanding file types using ls -l

“ - “ represent command-line options, standard input/output redirection, or directory shortcuts different things based on where it is used.

“ d “ represent as directory

In dev “ c “ represent character devices and “ b “ represent block devices

```

ibab@IBAB-Workshop-Comp15:~/lab2$ ls -l
total 12
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:28 data
drwxrwxr-x 2 ibab ibab 4096 Sep 22 11:29 files
drwxrwxr-x 2 ibab ibab 4096 Sep 21 17:30 output

```

```

ibab@IBAB-Workshop-Comp15:~/lab2$ cd /dev
ibab@IBAB-Workshop-Comp15:/dev$ pwd
/dev
ibab@IBAB-Workshop-Comp15:/dev$ ls -l
total 0
crw----- 1 root root 10, 262 Sep 23 08:28 acpi_thermal_rel
crw-r--r-- 1 root root 10, 235 Sep 23 08:28 autofs
drwxr-xr-x 2 root root 460 Sep 23 09:03 block
crw----- 1 root root 10, 234 Sep 23 08:28 btrfs-control
drwxr-xr-x 3 root root 60 Sep 23 08:28 bus
drwxr-xr-x 2 root root 4860 Sep 23 10:59 char
crw----- 1 root tty 5, 1 Sep 23 08:28 console
lrwxrwxrwx 1 root root 11 Sep 23 08:28 core -> /proc/kcore
drwxr-xr-x 22 root root 440 Sep 23 08:28 cpu
crw----- 1 root root 10, 260 Sep 23 08:28 cpu_dma_latency
crw----- 1 root root 10, 261 Sep 23 08:28 cpu_wakeup_latency
crw----- 1 root root 10, 203 Sep 23 08:28 cuse
drwxr-xr-x 10 root root 200 Sep 23 08:28 disk
drwxr-xr-x 2 root root 60 Sep 23 08:28 dma_heap
drwxr-xr-x 3 root root 100 Sep 23 08:28 dri
crw----- 1 root root 234, 0 Sep 23 08:28 drm_dp_aux0
crw----- 1 root root 234, 1 Sep 23 08:28 drm_dp_aux1
crw----- 1 root root 10, 258 Sep 23 08:28 ecryptfs
crw-rw---- 1 root video 29, 0 Sep 23 08:28 fb0
lrwxrwxrwx 1 root root 13 Sep 23 08:28 fd -> /proc/self/fd
crw-rw-rw- 1 root root 1, 7 Sep 23 08:28 full
crw-rw-rw- 1 root root 10, 229 Sep 23 08:28 fuse
crw----- 1 root root 254, 0 Sep 23 08:28 gpiochip0
crw----- 1 root root 240, 0 Sep 23 08:28 hidraw0
crw----- 1 root root 240, 1 Sep 23 08:28 hidraw1
crw----- 1 root root 240, 2 Sep 23 08:28 hidraw2
crw----- 1 root root 10, 228 Sep 23 08:28 hpet

```

```

brw-rw---- 1 root disk 7, 0 Sep 23 08:28 loop0
brw-rw---- 1 root disk 7, 1 Sep 23 08:28 loop1
brw-rw---- 1 root disk 7, 10 Sep 23 08:28 loop10
brw-rw---- 1 root disk 7, 11 Sep 23 08:28 loop11
brw-rw---- 1 root disk 7, 12 Sep 23 08:28 loop12
brw-rw---- 1 root disk 7, 2 Sep 23 08:28 loop2
brw-rw---- 1 root disk 7, 3 Sep 23 08:28 loop3
brw-rw---- 1 root disk 7, 4 Sep 23 08:28 loop4
brw-rw---- 1 root disk 7, 5 Sep 23 08:28 loop5
brw-rw---- 1 root disk 7, 6 Sep 23 08:28 loop6
brw-rw---- 1 root disk 7, 7 Sep 23 08:28 loop7
brw-rw---- 1 root disk 7, 8 Sep 23 08:28 loop8
brw-rw---- 1 root disk 7, 9 Sep 23 08:28 loop9

```

(14) Getting help about Linux commands

- ❖ “man” command is used for manual
- ❖ “-- help “ option provide a concise, built-in summary of a command's usage, syntax, and available arguments directly within the terminal screen
- ❖ Info command is available in my system

man ls

The image shows a terminal window titled "ibab@IBAB-Workshop-Comp15: ~ -- man ls". The terminal displays the help text for the 'ls' command, organized into sections: NAME, SYNOPSIS, DESCRIPTION, and OPTIONS. The OPTIONS section lists various flags and their functions, such as --help, --format, -C, -l, --long, -x, -T, --tabsize, -m, --zero, --dired, --hyperlink, -s, -o, -p, --numeric-uid-gid, --quoting-style, -M, --literal, -b, --escape, -Q, --quote-name, --hide-control-chars, --show-control-chars, --time, -c, -u, --hide, -T, --ignore, -a, --ignore-backups, --sort, -S, -t, -v, -X, -U, -k, --dereference, --dereference-command-line-symlink-to-dir, --hide-command-line, -G, --no-group, --author, -a, --all, -A, --almost-all, -f, -d, --directory, -h, --human-readable, -k, --kibibytes, --si, --block-size, -i, --inode, -r, --reverse, -R, --recursive, -w, --width, -s, --size, --color, --indicator-style, -F, --classify, --file-type, -p, --time-style, --full-time, -Z, --context, --group-directories-first, -V, --version, and [paths].

```
ibab@IBAB-Workshop-Comp15: ~ -- man ls
ls(1)
NAME
    ls - List directory contents. Ignore files and directories starting with a '.' by default

SYNOPSIS
    ls [-help] [-format] [-C] [-l|--long] [-x] [-T|--tabsize] [-m] [--zero] [-D|--dired] [--hyperlink] [-s] [-o] [-p] [--numeric-uid-gid] [--quoting-style] [-M|--literal] [-b|--escape] [-Q|--quote-name]
    [-q|--hide-control-chars] [--show-control-chars] [--time] [-c] [-u] [--hide] [-T|--ignore] [-a] [--ignore-backups] [--sort] [-S] [-t] [-v] [-X] [-U] [-k] [--dereference] [--dereference-command-line-symlink-to-dir]
    [--hide-command-line] [-G|--no-group] [--author] [-a] [--all] [-A] [--almost-all] [-f] [-d] [--directory] [-h] [--human-readable] [-k] [--kibibytes] [--si] [--block-size] [-i] [--inode] [-r] [--reverse] [-R]
    [--recursive] [-w] [--width] [-s] [--size] [--color] [--indicator-style] [-F] [--classify] [--file-type] [-p] [--time-style] [--full-time] [-Z] [--context] [--group-directories-first] [-V] [--version] [paths]

DESCRIPTION
    List directory contents. Ignore files and directories starting with a '.' by default

OPTIONS
    --help Print help information.

    --format
        Set the display format.

    -C
        Display the files in columns.

    -l, --long
        Display detailed information.

    -x
        List entries in rows instead of in columns.

    -T, --tabsize <COLS>
        Assume tab stops at each COLS instead of 8

    -m
        List entries separated by commas.

    --zero
        List entries separated by ASCII NUL characters.

    -D, --dired
        generate output designed for Emacs' dired (Directory Editor) mode

    --hyperlink[=WHEN] [default: never]
        hyperlink file names WHEN

        Possible values:
        * always
        * auto
        * never

    -1
        List one file per line.

    -o
        Long format without group information. Identical to --format=long with --no-group.
```

Manual page ls(1) line 1 (press h for help or q to quit)

man grep

```

ibab@IBAB-Workshop-Comp15: -- man grep
ibab@IBAB-Workshop-Comp15: ~/Downloads/pycharm-2026.2.3
ibab@IBAB-Workshop-Comp15: --

grep(1)
NAME
    grep, egrep, fgrep, rgrep - print lines that match patterns

SYNOPSIS
    grep [OPTION]... PATTERNS [FILE]...
    grep [OPTION]... -e PATTERNS ... [FILE]...
    grep [OPTION]... -f PATTERN_FILE ... [FILE]...

DESCRIPTION
    grep searches for patterns in each FILE. In the synopsis's first form, which is used if no -e or -f options are present, the first operand PATTERNS is one or more patterns separated by newline characters, and grep prints each line that matches a pattern. Typically PATTERNS should be quoted when grep is used in a shell command.

    A FILE of "-" stands for standard input. If no FILE is given, recursive searches examine the working directory, and nonrecursive searches read standard input.

    Debian also includes the variant programs egrep, fgrep and rgrep. These programs are the same as grep -E, grep -F, and grep -r, respectively. These variants are deprecated upstream, but Debian provides for backward compatibility. For portability reasons, it is recommended to avoid the variant programs, and use grep with the related option instead.

OPTIONS
    Generic Program Information
    --help Output a usage message and exit.

    -V, --version
        Output the version number of grep and exit.

    Pattern Syntax
    -E, --extended-regexp
        Interpret PATTERNS as extended regular expressions (ERES, see below).

    -F, --fixed-strings
        Interpret PATTERNS as fixed strings, not regular expressions.

    -G, --basic-regexp
        Interpret PATTERNS as basic regular expressions (BRES, see below). This is the default.

    -P, --perl-regexp
        Interpret PATTERNS as Perl-compatible regular expressions (PCRES). This option is experimental when combined with the -z (--null-data) option, and grep -P may warn of unimplemented features.

    Matching Control
    -e PATTERNS, --regexp=PATTERNS
        Use PATTERNS as the patterns. If this option is used multiple times or is combined with the -f (--file) option, search for all patterns given. This option can be used to protect a pattern beginning with "-".

    -f FILE, --file=FILE
        Obtain patterns from FILE, one per line. If this option is used multiple times or is combined with the -e (--regexp) option, search for all patterns given. The empty file contains zero patterns, and therefore matches nothing. If FILE is -, read patterns from standard input.

    -i, --ignore-case
        Ignore case distinctions in patterns and input data, so that characters that differ only in case match each other.

    --no-ignore-case

Manual page grep(1) line 1 (press h for help or q to quit)
  
```

ls -help

```
Sep 24 10:00
ibab@IBAB-Workshop-Comp15: ~
ibab@IBAB-Workshop-Comp15: ~
ibab@IBAB-Workshop-Comp15: ~/Downloads/pycharm-2026.2.3
ibab@IBAB-Workshop-Comp15: ~

-L, --dereference                When showing file information for a symbolic link, show information for the
                                file the link references rather than the link itself.
--dereference-command-line-symlink-to-dir Do not follow symlinks except when they link to directories and are
                                given as command line arguments.
-H, --dereference-command-line Do not follow symlinks except when given as command line arguments.
-G, --no-group                    Do not show group in long format.
-a, --all                        Show author in long format. On the supported platforms,
                                the author always matches the file owner.
-A, --almost-all                Do not ignore hidden files (files with names that start with '.').
                                In a directory, do not ignore all file names that start with '.',
                                only ignore '.' and '..'.
-f                               List all files in directory order, unsorted. Equivalent to -AU. Disables --color unless explicitly specified.
--directory                     Only list the names of directories, rather than listing directory contents.
                                This will not follow symbolic links unless one of --dereference-command-line
                                (-H), --dereference (-L), or --dereference-command-line-symlink-to-dir is
                                specified.
-h, --human-readable             Print human readable file sizes (e.g. 3K 230M 56G).
-k, --kibibytes                 default to 1024-byte blocks for file system usage; used only with -s and per
                                directory totals.
--si                            Print human readable file sizes using powers of 1000 instead of 1024.
--block-size=BLOCK_SIZE        scale sizes by BLOCK_SIZE when printing them
-i                               print the index number of each file
-l, --inode                     Reverse whatever the sorting method is e.g., list files in reverse
-r, --reverse                   alphabetical order, youngest first, smallest first, or whatever.
-R, --recursive                 List the contents of all directories recursively.
-w, --width <COLS>             Assume that the terminal is COLS columns wide.
-s, --size                       print the allocated size of each file, in blocks
--color[=<color>]               Color output based on file type. [possible values: always, auto, never]
--indicator-style <indicator-style> Append indicator with style X000 to entry names:
                                none (default), slash (/), file-type (--file-type), classify (-f) [possible values: none, slash, file-type, classify]
-f, --classify[=<when>]         none (default), slash (/), file-type (--file-type), classify (-f) [possible values: none, slash, file-type, classify]
                                Append a character to each file name indicating the file type. Also, for
                                regular files that are executable, append 'x'. The file type indicators are
                                '/' for directories, 'l' for symbolic links, 'p' for FIFOs, '=' for sockets,
                                'd' for doors, and nothing for regular files, when may be omitted, or one of:
                                none - Do not classify. This is the default.
                                auto - Only classify if standard output is a terminal.
                                always - Always classify.
                                Specifying --classify and no when is equivalent to --classify=always. This will
                                not follow symbolic links listed on the command line unless the
                                --dereference-command-line (-H), --dereference (-L), or
                                --dereference-command-line-symlink-to-dir options are specified. [possible values: always, auto, never]
                                Same as --classify, but do not append 'x'
--file-type                     Append / indicator to directories.
-p, --time-style <TIME_STYLE>   time/date format with %i; see TIME_STYLE below [env: TIME_STYLE=]
--full-time                     like -l --time-style=full-iso
-Z, --context                   print any security context of each file
                                group directories before files; can be augmented with
                                a --sort option, but any use of --sort=none (-U) disables grouping
-V, --version                   Print version

The TIME_STYLE argument can be full-iso, long-iso, iso, locale or +FORMAT. FORMAT is interpreted like in date. Also the TIME_STYLE environment variable sets the default style to use.
ibab@IBAB-Workshop-Comp15: ~
```

grep -help

```
ibab@IBAB-Workshop-Comp15:~$ grep --help
Usage: grep [OPTION]... PATTERNS [FILE]...
Search for PATTERNS in each FILE.
Example: grep -i 'hello world' menu.h main.c
PATTERNS can contain multiple patterns separated by newlines.

Pattern selection and interpretation:
-E, --extended-regexp    PATTERNS are extended regular expressions
-F, --fixed-strings       PATTERNS are strings
-G, --basic-regexp        PATTERNS are basic regular expressions
-P, --perl-regexp         PATTERNS are Perl regular expressions
-e, --regexp=PATTERNS     use PATTERNS for matching
-f, --file=FILE           take PATTERNS from FILE
-i, --ignore-case         ignore case distinctions in patterns and data
--no-ignore-case          do not ignore case distinctions (default)
-w, --word-regexp         match only whole words
-x, --line-regexp         match only whole lines
-z, --null-data           a data line ends in 0 byte, not newline

Miscellaneous:
-s, --no-messages        suppress error messages
-v, --invert-match        select non-matching lines
-V, --version             display version information and exit
-h, --help                display this help text and exit

Output control:
-m, --max-count=NUM       stop after NUM selected lines
-b, --byte-offset         print the byte offset with output lines
-n, --line-number         print line number with output lines
--line-buffered           flush output on every line
-H, --with-filename       print file name with output lines
-h, --no-filename         suppress the file name prefix on output
--label=LABEL            use LABEL as the standard input file name prefix
-o, --only-matching       show only nonempty parts of lines that match
-q, --quiet, --silent     suppress all normal output
--binary-files=TYPE       assume that binary files are TYPE;
                           TYPE is 'binary', 'text', or 'without-match'
-a, --text                equivalent to --binary-files=text
-I                         equivalent to --binary-files=without-match
-d, --directories=ACTION   how to handle directories;
                           ACTION is 'read', 'recurse', or 'skip'
-D, --devices=ACTION     how to handle devices, FIFOs and sockets;
                           ACTION is 'read' or 'skip'
-r, --recursive           like --directories=recurse
-R, --dereference-recursive likewise, but follow all symlinks
--include=GLOB            search only files that match GLOB (a file pattern)
--exclude=GLOB            skip files that match GLOB
--exclude-from=FILE       skip files that match any file pattern from FILE
--exclude-dir=GLOB        skip directories that match GLOB
-L, --files-without-match print only names of FILES with no selected lines
-l, --files-with-matches  print only names of FILES with selected lines
-c, --count               print only a count of selected lines per FILE
```

10.1 'ls': List directory contents

=====

The 'ls' program lists information about files (of any type, including directories). Options and file arguments can be intermixed arbitrarily, as usual. Later options override earlier options that are incompatible.

For non-option command-line arguments that are directories, by default 'ls' lists the contents of directories, not recursively, and omitting files with names beginning with '.'. For other non-option arguments, by default 'ls' lists just the file name. If no non-option argument is specified, 'ls' operates on the current directory, acting as if it had been invoked with a single argument of '.'.

By default, 'ls' lists each directory's contents alphabetically, according to the locale settings in effect.⁽¹⁾ If standard output is a terminal, the output is in columns (sorted vertically) and control characters are output as question marks; otherwise, the output is listed one per line and control characters are output as-is.

Because 'ls' is such a fundamental program, it has accumulated many options over the years. They are described in the subsections below; within each section, options are listed alphabetically (ignoring case). The division of options into the subsections is not absolute, since some options affect more than one aspect of 'ls''s operation.

Exit status:

- 0 success
- 1 minor problems (e.g., failure to access a file or directory not specified as a command line argument. This happens when listing a directory in which entries are actively being removed or renamed.)
- 2 serious trouble (e.g., memory exhausted, invalid option, failure to access a file or directory specified as a command line argument or a directory loop)

Also see [*note Common options::](#).

* Menu:

- * [Which files are listed::](#)
- * [What information is listed::](#)
- * [Sorting the output::](#)
- * [General output formatting::](#)
- * [Formatting file timestamps::](#)
- * [Formatting the file names::](#)

----- Footnotes -----

-----Info: (coreutils)ls invocation, 56 lines --Top-----

(15) Binary representation of numbers

```
ibab@IBAB-Workshop-Comp15:~/lab1/Q10/q1$ bc
bc 1.07.1
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2011
This is free software with ABSOLUTELY NO WARRANTY.
For details type `warranty'.
ibase=10
obase=2
2
10
3
11
4
100
5
101
6
110
8
1000
10
1010
15
1111
16
10000
20
10100
25
11001
```

```
ibab@IBAB-Workshop-Comp15:~/lab1/Q10/q1$ bc
bc 1.07.1
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2011
This is free software with ABSOLUTELY NO WARRANTY.
For details type `warranty'.
obase=10
ibase=2
101
5
1010
10
1111
15
10000
16
10101
21
11001
25
```

Decimal	Binary
0	1
1	2
2	10
3	11
4	100
5	101
6	110
7	111
8	1000

9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	10000
16	10001

- Using given bits largest decimal number that can be represented is 255
- Using 4 bits 16 different values can be represented
- Using 8 bits 255 different values can be represented

Calculation $2^8 - 1 = 255$

(16) Binary representation using Linux

```
ibab@IBAB-Workshop-Comp15:~/lab1/Q10/q1$ bc
bc 1.07.1
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2
This is free software with ABSOLUTELY NO WARRANTY.
For details type 'warranty'.
ibase=10
obase=2
10
1010
15
1111
20
10100
25
11001
quit
ibab@IBAB-Workshop-Comp15:~/lab1/Q10/q1$ bc
bc 1.07.1
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2
This is free software with ABSOLUTELY NO WARRANTY.
For details type 'warranty'.
obase=10
ibase=2
1010
10
1111
15
10100
20
11001
25
```

The binary operation and decimal outputs are same each time

(17) Final command-line exercise

1. "ls" command helps to list all the files in the directory

```

ibab@IBAB-Workshop-Comp15:~$ ls
2      Documents  Pictures      R      apt.conf      lab2      output      sails_python_labs  testdir-hardlink
Desktop Downloads    Public       Templates  commands_output.txt  file1_hardlink  lab      listing.txt      overwrite.txt      snap      testdir-synlink
Docs   Music       PycharmProjects  Videos   data          files         lab1     location.txt     pushtestdir      testdata.csv  textfiles.txt

ibab@IBAB-Workshop-Comp15:~$ ls -l
total 128
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 2
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Desktop
drwxr-xr-x 5 ibab ibab 4096 Sep 18 12:30 Docs
drwxr-xr-x 5 ibab ibab 4096 Sep 18 10:43 Documents
drwxr-xr-x 3 ibab ibab 4096 Sep 23 11:26 Downloads
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Music
drwxr-xr-x 3 ibab ibab 4096 Sep 17 14:38 Pictures
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Public
drwxrwxr-x 3 ibab ibab 4096 Sep 23 11:34 PycharmProjects
drwxrwxr-x 4 ibab ibab 4096 Sep 24 15:39 R
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Templates
drwxr-xr-x 2 ibab ibab 4096 Sep 17 14:07 Videos
-rw-rw-r-- 1 ibab ibab 279 Sep 17 14:25 apt.conf
-rw-rw-r-- 1 ibab ibab 314 Sep 22 12:32 commands_output.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 data
-rw-rw-r-- 1 ibab ibab 209 Sep 22 11:38 directory.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 18 13:01 file1_hardlink
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:25 files
-rw-rw-r-- 1 ibab ibab 48 Sep 22 11:56 information.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 18 10:35 lab
drwxrwxr-x 3 ibab ibab 4096 Sep 24 16:03 lab1
drwxrwxr-x 5 ibab ibab 4096 Sep 21 14:51 lab2
-rw-rw-r-- 1 ibab ibab 1318 Sep 22 11:39 listing.txt
-rw-rw-r-- 1 ibab ibab 11 Sep 22 11:39 location.txt
drwxrwxr-x 2 ibab ibab 4096 Sep 21 14:24 output
-rw-rw-r-- 1 ibab ibab 1374 Sep 22 11:50 overwrite.txt
drwxrwxr-x 3 ibab ibab 4096 Sep 21 15:34 pushtestdir
drwxrwxr-x 4 ibab ibab 4096 Sep 24 13:15 sails_python_labs

```

2. `cat <file name>` display all the contents of a file

```
l1ab@l1AB-Workshop-Comp15:~/lab2/files$ cat Sample.txt file.txt file2.txt file3.txt file_copy.txt second.txt sorted_Sample.txt
The ribosome distinguishes the correct initiation AUG from internal methionine AUGs by relying on the molecular context surrounding the codon. This mechanism differs f
undamentally between prokaryotes and eukaryotes.
Prokaryotes (Bacteria)

In bacteria, the ribosome identifies the start codon through direct RNA-to-RNA base pairing just upstream of the AUG.

The Shine-Dalgarno (SD) Sequence: The mRNA transcript contains a specific purine-rich consensus sequence (typically AGGAGG) located about 8 to 10 nucleotides upstr
eam (5' side) of the start codon.

16S rRNA Pairing: The small ribosomal subunit (30S) contains 16S ribosomal RNA, which possesses a complementary sequence at its 3' end (the anti-Shine-Dalgarno seq
uence).

Mechanical Alignment: When the 16S rRNA hydrogen-bonds to the Shine-Dalgarno sequence on the mRNA, it physically anchors and aligns the ribosome so that the downst
ream AUG codon drops perfectly into the ribosome's P-site, ready for the initiator tRNA (fMet-tRNA) to bind.
Eukaryotes (Plants and Animals)
```

- ### 3. grep

```

lab@IBAB-Workshop-Comp15:~/lab2/files$ grep -i "glycine" Sample.txt
The Break: When the ribosome attempts to form a peptide bond between the final glycine (G) and the final Proline (P) of the 2A consensus sequence, the bond physically cannot form due to the distorted geometry.
Release and Restart: Because the bond fails, Protein A is released from the ribosome (carrying most of the 2A tag, ending in glycine, attached to its end). The ribosome, however, does not fall off the mRNA. It keeps the Proline-tRNA in place and continues translating the downstream sequence, making Protein B (which now starts with that Proline).

```

4. `wc -w <file name>`

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ wc -w Sample.txt
618 Sample.txt
```

5. `sort input file name .txt > outputfile name .txt`

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ sort Sample.txt > sorted_Sample2.txt
```

6. `echo input file name.txt >> outputfile name.txt`

```
Workshop-Comp15:~/lab2/files$ echo "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa" >> file.txt
Workshop-Comp15:~/lab2/files$ cat file.txt
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

```
Translation: The ribosome translates the first gene (Protein A) and moves directly into the 2A sequence, synthesizing a long polypeptide chain containing both protein sequences.
```

7. `sort -i <file name>`

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ sort Sample.txt

16S rRNA Pairing: The small ribosomal subunit (30S) contains 16S
ence).
5' Cap Recognition: The small ribosomal subunit (40S), already ca
cap structure (m7G) at the extreme 5' end of the mRNA.
All functional 2A peptides share a conserved structural motif at thei
Eukaryotes (Plants and Animals)
Eukaryotic ribosomes do not use a Shine-Dalgarno sequence. Instead, t
ed of the mRNA transcript
```

8. Find is used to locate the particular file

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ cat file.txt | wc -w
7
ibab@IBAB-Workshop-Comp15:~/lab2/files$ find Sample.txt
Sample.txt
```

9. `ls -l` gives all the information of the files

```
ibab@IBAB-Workshop-Comp15:~/lab2/files$ ls -l
total 24
-rw-rw-r-- 1 ibab ibab 4145 Sep 21 14:51 Sample.txt
-rw-rw-r-- 1 ibab ibab 4285 Sep 24 18:22 file.txt
-rw-rw-r-- 1 ibab ibab  0 Sep 21 14:31 file2.txt
-rw-rw-r-- 1 ibab ibab  0 Sep 21 14:31 file3.txt
-rw-rw-r-- 1 ibab ibab  0 Sep 21 17:27 file_copy.txt
-rw-rw-r-- 1 ibab ibab  0 Sep 21 17:24 outout
-rw-rw-r-- 1 ibab ibab  0 Sep 21 17:25 output
-rw-rw-r-- 1 ibab ibab  98 Sep 22 11:29 second.txt
-rw-rw-r-- 1 ibab ibab 868 Sep 24 18:22 sorted_Sample2.txt
```